

RegEx, Requests, BeautifulSoup

This document contains useful information on how to retrieve and parse data from the web.

Note: The terminology included in this document and a basic understanding of regex, requests, and BeautifulSoup are course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction	2
Terminology	2
Regular Expressions	3
0-Meta Characters	3
Literal Characters	3
Important Module Methods	4
Match Object Methods	4
Greedy vs. Non-Greedy	5
Backreferences	5
The Requests Module	6
requests.get	6
requests.post	6
response object methods and attributes	6
status_code	6
json	7
text	7
Additional parameters for a request	8
headers	8
Web Scraping	10
Scraping Ethics	10
Web Scraping with RegEx	10
Google Chrome Inspect Tool	11
HTML Rendering	11
Beautiful Soup	12
Constructing a BeautifulSoup Object	12
Using a BeautifulSoup Object	12
Putting It All Together: Web Scraping Example	13
Selenium	15

Introduction

In order to begin parsing data using regular expressions and retrieving data from the web, we need to establish common ground regarding the terminology and software we will be using in this course.

Terminology

Web - The World Wide Web. For our purposes, any webpage with interesting or useful data that we are allowed to access.

Scraping - Extracting data from a webpage for later use or analysis. In this class, scraping will typically be done with the “requests” module and either the regex (Regular Expressions) module, the “BeautifulSoup” module, or both.

Bot - A computer program acting on behalf of a user to automatically scrape the web.

robots.txt - A text file that some websites have to specify which portions (if any) of their site can be accessed by a bot.

HTTP - HyperText Transfer Protocol, the system used by the World Wide Web that defines how messages are formatted and transmitted between sites.

get request - One of the most common HTTP methods. Used to retrieve data from a website. With the requests module, a get request is accessed by invoking `requests.get()`. Do not use it for sending sensitive data (like passwords).

post request - A less common HTTP method, typically used for uploading or submitting data to a website. With the requests module, a post request is accessed by invoking `requests.post()`. Used when sensitive data needs to be sent.

parse - The process of analyzing a string so that we can understand its content and thus access its data more easily.

API - Application Programming Interface. Used to easily access data hosted somewhere else that might frequently change or have restricted access. The response is typically JSON.

response - The server’s response to the HTTP request. The requests module is represented by the response object, upon which there are many possible methods we can call to get more info about the response.

Regular Expressions

A Regular Expression is a sequence of symbols and characters expressing a particular string or pattern to be searched for within a longer piece of text. This sequence is typically a combination of metacharacters, character classes, and/or literal characters (e.g. 'a.a', 'Kat[a-z]', '.*?'). Regular Expressions are very powerful as they can be used to match complex patterns within long strings. They can be used in web scraping to extract useful data from lengthy HTML strings. *Remember to **import re!!***

0-Meta Characters

A character used in a regular expression has a special meaning. Can be used to match very complicated strings (see table below). Use a “\” to turn a metacharacter into a literal character.

Literal Characters

Any character that is not a metacharacter. Examples include letters, numbers, #, etc. Will match the character itself.

Meta Character	Meaning
.	Matches any character
\	Escape special/meta characters
	Or operator
^	Match at beginning of string/line. Represents “not” in a character class
\$	Match at end of string/line
*	Match 0 or more of the preceding regex
+	Match 1 or more of the preceding regex
?	Match 0 or 1 of the preceding regex
{ }	Bounded repetition
[]	Create a character class
()	Capture group within the matched substring

Character Class	What it matches
[a-z]	Any lowercase letter
[A-Z]	Any uppercase letter
[a-zA-Z]	Any letter
[^A-Z]	Anything except uppercase letters

Predefined character class	What it matches
\d	Any digit, equivalent to [0-9]
\D	Any non-digit, equivalent to [^0-9]
\s	Any whitespace char, equal to [\t\n\r\f\v]
\S	Any non-whitespace char, equal to [^\t\n\r\f\v]
\w	Any alphanumeric char, equal to [a-zA-Z0-9_]
\W	Any non alphanumeric char, equal to [^a-zA-Z0-9_]

Important Module Methods

All of the following methods take in a regular expression inside of a raw string (denoted by the “r” before the string) as their first parameter, and a string to search for matches to the regular expressions as another parameter (*a_string*).

Raw string application example: r“\n”, a raw string, is a string of two characters “\” and “n”, while “\n” is a normal string of one character, the new line character.

Module Method	What it does
<code>re.match(r'regex', a_string)</code>	Checks if the beginning of <i>a_string</i> matches the regex given in the first argument. If so, it returns a Match object. Otherwise, it returns None .
<code>re.search(r'regex', a_string)</code>	Checks if any part of <i>a_string</i> matches the regex given in the first argument. If so it returns a Match object corresponding to the first matching part. If it doesn't match at all, returns None .
<code>re.findall(r'regex', a_string)</code>	Checks <i>a_string</i> for all non-overlapping matches to the regex supplied and returns a list of the matching portions of <i>a_string</i> .
<code>re.sub(r'regex', new_string, a_string)</code>	Checks <i>a_string</i> for all matches to the regex and returns a string with each match replaced by <i>new_string</i>

Match Object Methods

You may have noticed that many of the RegEx methods return a special Match Object. We will not cover these in-depth in this course, however, it is useful to know a couple of the Match Object methods.

Method	What it does
<code>match_object.start()</code>	Returns the index of the start of the string that matched the regex.
<code>match_object.end()</code>	Returns the index after the end of the string that matched the regex.
<code>match_object.group()</code>	Returns the string that matched the regex
<code>match_object.span()</code>	Returns a tuple of the starting and ending indices of the string matched by the regex *Ending index is exclusive

To see all of the match object methods, or to learn more about match objects, visit the official python documentation for RegEx: <https://docs.python.org/3.7/library/re.html#match-objects>

Greedy vs. Non-Greedy

By default, regular expressions are greedy, meaning they will match the **longest** string possible. We can make a regex non-greedy by putting a **?** after the **+** or ***** meta characters. This will make them match the **shortest** string possible instead.

Basic RegEx Example:

```
>>> import re
>>> a_str = "buzzBuzzbuzz"
>>> matching_str = re.search(r"b.*z", a_str) ##this is greedy
>>> matching_str.group()
"buzzBuzzbuzz"
>>> matching_str2 = re.search(r"B.*z", a_str) ##also greedy
>>> matching_str2.group()
"Buzzbuzz"
>>> matching_str3 = re.search(r"b.*?z", a_str) ##non-greedy
>>> matching_str3.group()
'buz'
```

Backreferences

Backreferences will match the same text that was previously matched in a capture group. It is symbolized with `"\N"`, where N is a number referring to the capture group. You will need to specify which capture group you are referencing if you have more than 1. To count capture groups, count from left to right. The first opening parentheses refers to the start of the first capture group, the second opening parentheses refers to the start of the second capture group, and so on.

Note: there may be discrepancies in how backreferences work with `.findall()`. It works best with `.search()` and `.match()`.

For `.findall()`, backreferences will return the **first** capture case (the backreference) and any other capture cases it can find in a **list of tuples**.

Example: return the first instance of the string containing two numbers in a row.

```
>>> statement = "he11o everyone, a11 is we11 here"
>>> print(re.search(r"([a-z]*([0-9])\2[a-z]*)", statement).group())
>>> he11o
```

The Requests Module

The first step to receiving information from the web is to request it. This is done through the requests module. *Remember to **import requests!!***

requests.get

The most common method we will use in this class. Returns a response object that we can access to view, clean, and analyze the requested data.

Usage:

```
response = requests.get("https://example.com")
```

requests.post

requests.post allows our program to send information to a given webpage, an example of this is filling out a form to log into an account. This will **not** be tested in this class but could be useful for some applications.

Usage:

```
response = requests.post("https://example.com", data={"some": "data"})
```

response object methods and attributes

After a successful get or post request, a response object contains information that can be used within your code. There are 2 attributes and 1 method that we will most often use involving these response objects in CS2316: `response.status_code`, `response.text`, and `response.json()`.

status_code

`response.status_code` gives us an int that represents the status of the request. There are many possible status codes, but some useful codes include:

200: Successful request.

404: URL not found. Most likely indicates that an incorrect URL was given to the get request.

500: Internal Server Error. This is a general catch-all which could indicate issues with connection to the server.

401/403: Unauthorized/Forbidden. You have tried to access a URL that you do not have access to.

Usage:

```
>>> resp = requests.get("http://gatech.edu")
>>> resp.status_code
200
```

json

The content of a response object can sometimes be in JSON format, most often when requesting from an API. This makes it very easy to process the data in the response object by simply calling the `response.json()` method, which converts the JSON data into a python dictionary.

Example: Consider the API linked at <http://open-notify.org/Open-Notify-API/ISS-Location-Now/>. This is a very basic API which provides info on the current location of the International Space Station. We can easily access the API's data using requests and the `response.json()` method.

```
>>> import requests
>>> resp = requests.get("http://open-notify.org/Open-Notify-API/ISS-Location-Now/")
>>> resp.json()
{"message": "success", "iss_position": {"latitude": "25.2991", "longitude":
"10.2294"}, "timestamp": 1578944691}
```

text

In cases where a site does not offer an API, the response will most likely not be in JSON format, but will instead contain the raw HTML of the website which we can access through `response.text`. To obtain useful data from the HTML, it is helpful to use the modules and methods discussed later in this handout. The `response.text` attribute returns one string of all of the HTML code inside a response object.

Example: Consider sending a request to <http://google.com>. This is not an API, so the response will contain a very long string containing the raw HTML of the site.

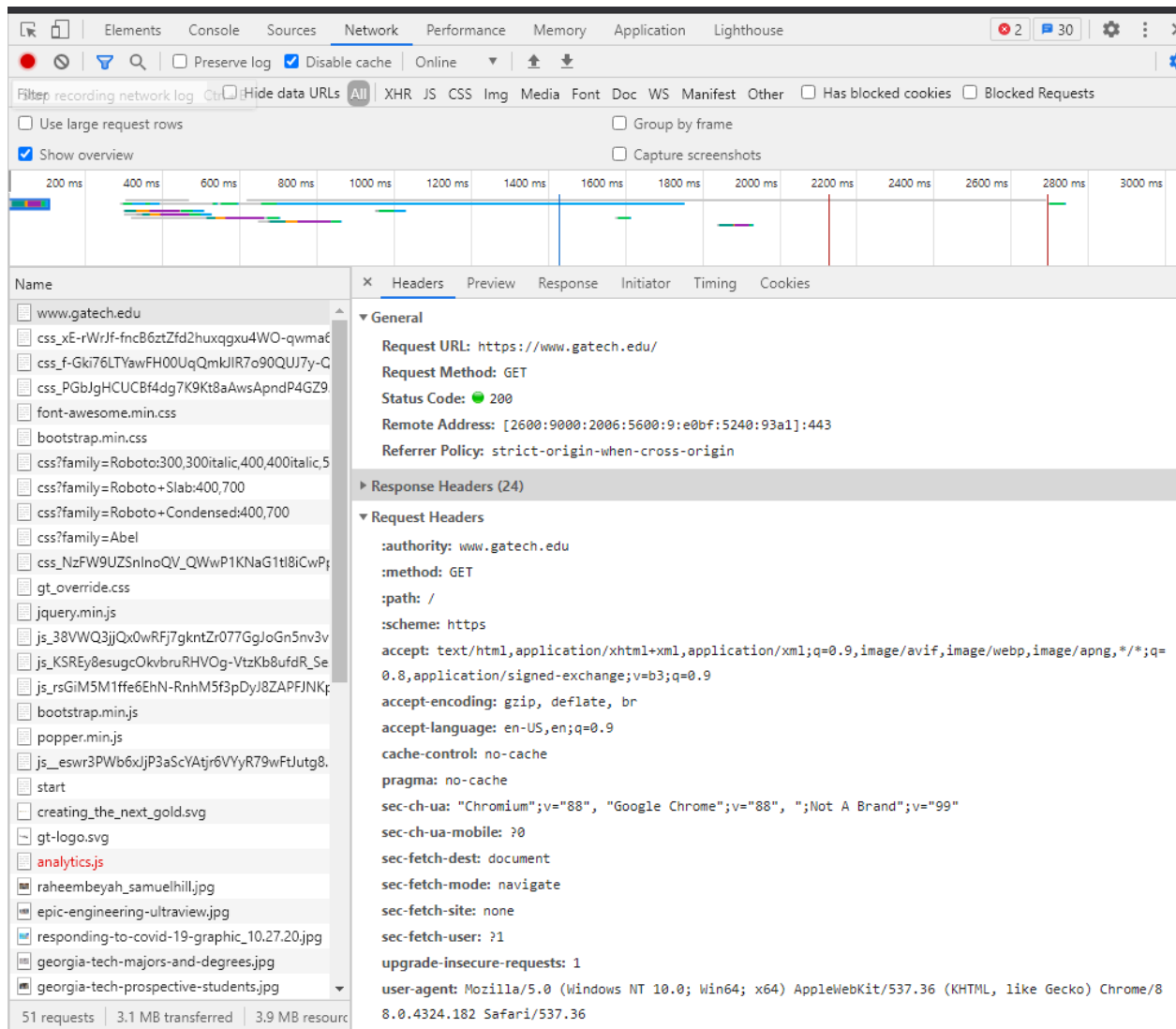
```
>>> resp = requests.get("http://google.com")
>>> resp.text
'<!doctype html><html itemscope= "" itemtype="http://schema.org/WebPage"
lang="en"><head><meta content="Search the world\'s information,
including webpages, images, videos and more. Google has many special
features to help you find exactly what you\'re looking for."
name="description"><meta content="noodp" name="robots"><meta
content="text/html; charset=UTF-8" http-equiv="Content-Type"> . . . . .
(13,000 more characters)
```

This screenshot only includes a very small portion of the HTML, so in order to extract anything meaningful from raw HTML, it is necessary to use either RegEx module or BeautifulSoup each which is described below.

Additional parameters for a request

headers

The “headers” of a request contain metadata about the request, such as the source of the request (such as what browser and operating system is sending the request). You can view the headers sent with a request in the developer tools view of your browser if you are interested.



Sometimes, when making a request programmatically, you will need to add or modify the headers of the request. A common header that is attached to requests sent by your browser is the “user-agent” header. In the image above, the “user-agent” header describes what kind of browser / device is sending the request. This header is one of the ways a website might detect that you are accessing the website from a mobile device. Based on this “user-agent” header, you may be shown the mobile version of the web page. Similarly, the requests module sends its own “user-agent”, which looks

something like 'python-requests/2.22.0'. Some websites may try to block or limit requests that do not come from a browser, or are specifically coming via a program. One such example is Reddit, which limits the number of requests able to be sent from the Python requests module. However, we can manually set the “user-agent” header to anything. Here is the format:

```
>>> url = 'https://www.gatech.edu/'  
>>> r = requests.get(url, headers={'user-agent': 'my-app/0.0.1'})
```

You can even make the request seem as if it is coming from your browser, by setting the user-agent to something like: “Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.182 Safari/537.36” (this user-agent string was taken from a browser request like shown in the image above).

An API may require you to use other headers (such as for Authorization). If the documentation of the API mentions the need to include headers with your request, you can use this headers parameter and pass in a dictionary just like shown above, with the header field name as the dictionary key, and the field value as the dictionary value.

Web Scraping

There is a lot of data available on the web, but it takes *some* work to get it in a form that can be used for analysis. Web scraping is the process of extracting data from a webpage for later use or analysis. It is needed because desirable data stored in web pages are not in a structured format convenient for analysis (like CSV and JSON files).

An important downside of web scraping is that web pages are frequently updated, and any code written for web scraping may break at any time. Though, if the web scraping is a one-time thing, then you do not need to worry about changes to the web site. If the web scraping needs to be done routinely, such as weekly, then consider investigating if there is an API available for the same data. API's are made for this very purpose of programmatically interacting with the website, and generally do not change as often. However, often there will not be an available API, which is when web scraping will be useful.

In this class, scraping will typically be done with the `requests` module, the `re` (Regular Expressions) module, the `BeautifulSoup` module, or combination of the three. Before we dive into examples where we parse data from the web, it is vital to understand the responsibility that comes with using these tools.

Scraping Ethics

While web scraping it's critical to be aware of the implications of your actions. Consider the following snippet of code:

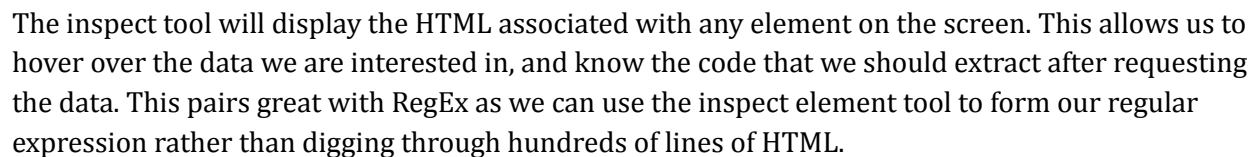
```
my_list = []
for i in range(10000):
    for j in range(1000):
        my_list.append(requests.get(f"http://dontdothis.com/{i}/{j}"))
```

How many requests is this sending to the site? What could this mean for the site's servers? To ensure you are operating within your legal limits, be sure to check for a **robots.txt**, and always use a website's API if one is offered. For example, see <https://atlanta.craigslist.org/robots.txt>

Web Scraping with RegEx

RegEx is a very powerful tool in web scraping. While BeautifulSoup, the module discussed later, is better for parsing through straightforward HTML, there are many situations in which the data we wish to extract is deep within HTML tags and cannot be directly extracted otherwise. Regular expressions can often be used to parse the data that you are looking for.

Google Chrome has a handy tool we can use to make our lives easier while web scraping. First click the three vertical dots in the upper right hand corner -> More tools -> Developer Tools. Alternatively, Ctrl+Shift+I on Windows and Cmd+Opt+J on Mac. Then select the square with a cursor in the top left of this menu.



There are two main rendering methods, static and dynamic. Static rendering is where all the content on the web page is received immediately on the first request sent (usually a GET request). This method is commonly seen in simpler web pages, and it is simple for us to scrape. This method is viable for certain use cases however many websites use different methods nowadays.

11

One intermediate solution for web scraping dynamic content is the Selenium library. It runs a real browser in the background but allows us to interact with the browser programmatically (so we can do things like wait for the page to fully load, click buttons, or fill out forms like a normal user). We will not cover it in this class, but you may find it useful when you find a website you wish to scrape dynamically rendered websites. You can identify that the website is dynamically rendered by comparing the HTML when you view the source (by right-clicking and selecting “view source”) to the HTML received when you send a GET request via the requests module. If the content is different, then it is highly likely that the content of the web page is dynamically rendered.

Beautiful Soup

“Beautiful Soup is a Python library for pulling data out of HTML and XML files. It works with your favorite parser to provide idiomatic ways of navigating, searching, and modifying the parse tree. It commonly saves programmers hours or days of work.” - Official BeautifulSoup documentation.

We can use BeautifulSoup (aka bs4) to parse through HTML files in a much easier way, saving us the trouble of making regular expressions.

*Remember to use **from bs4 import BeautifulSoup!!***

Constructing a BeautifulSoup Object

To begin parsing a file using bs4, we first construct a BeautifulSoup object. This object will have many useful methods that we can use to parse the file and extract data.

Usage:

```
>>> from bs4 import BeautifulSoup

>>> file_to_parse = "web_scraping.html"

>>> with open(file_to_parse) as f:

...     soup = BeautifulSoup(f, parser="html.parser")
```

This creates a BeautifulSoup object, which parses the text for us using built-in regular expressions. Notice that in the example above, the first argument we pass in to the BeautifulSoup constructor is a file object. This can also be a single string containing the HTML code. We pass in “html.parser” to denote that we want to parse HTML data.

Using a BeautifulSoup Object

There are many helpful ways that we can use a soup object to parse through HTML. Here are a few examples.

Method	What it does
soup.head	Returns a tag object representing the head tag
soup.body	Returns a tag object representing the <u>first</u> body tag
soup.find("tag")	Returns a tag object that represents the <u>first</u> instance of the tag that was input
soup.find_all("tag")	Returns a list of tag objects of <u>all</u> instances of the tag that was input
(Alternate Forms) soup.find("tag", {"tag-attribute" : "attribute-value"}) soup.find_all("tag", {"tag-attribute" : "attribute-value"})	Same as `find` and `find_all`, but can filter the returned "tag"s by specific attribute(s) using the optional second argument (expects a dictionary) To find , we can use either of the following: <pre>soup.find_all("a", {"class" : "big"}) soup.find_all("a", {"class" : "big", "xd": "seven"})</pre>

Once we have a Tag object, we can access the information pertaining to that tag.

Method	What it does
tag.text	Returns a string containing the text displayed by a given tag
tag["attrib"]	Allows us to access a tag's attributes by treating the Tag object as if it were a dictionary with the attribute as the key.

For more information, visit <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Putting It All Together: Web Scraping Example

Let's say we were interested in creating a program to help us monitor the price of a 24 pack of the beef flavored instant ramen sold at:

<https://www.roundeyesupply.com/Ramen-Noodle-Soup-s/948.htm>.

We can accomplish this using everything we have learned in this handout! We begin with requests:

```
import requests, re
from bs4 import BeautifulSoup
url = "https://www.roundeyesupply.com/Ramen-Noodle-Soup-s/948.htm"
resp = requests.get(url)
```

Now that we have our response object, we can plug the HTML of the website into the BeautifulSoup constructor using `resp.text` (recall that `resp.text` gives us a string containing all the HTML code):

```
soup = BeautifulSoup(resp.text, "html.parser")
```

Now, using the Google Chrome Inspect Element tool, we can identify that the classes of interest for product name and product price are `"v-product_title"` and `"product_productprice"` respectively.

```
<a href="//www.roundeyesupply.com/Maruchan Ramen Beef Flavor 3 oz. Noodle Soup-p/DE487096.htm" class="v-product_title productnamecolor colors_productname ng-binding" title="Maruchan Ramen Beef Flavor 3 oz. Noodle Soup" intellisuggest ss-ps>Maruchan Ramen Beef Flavor 3 oz. Noodle Soup</a> == $0
```

```
<div class="product_productprice">
  <!-- ngIf: (result.price * 1) > 0 && result.ss_sku_field != 'GFT' -->
  <span ng-if="(result.price * 1) > 0 && result.ss_sku_field != 'GFT'" ng-class="{product_saleprice':
```

We can use BeautifulSoup to create lists of these tags, and then convert them to strings using `tag.text`:

```
>>> titles = soup.find_all(class_ = "v-product_title")
>>> cleaned_titles = [title.text for title in titles]
>>> cleaned_titles
["\r\nMaruchan Ramen Beef Flavor 3 oz. Noodle Soup\r\n",
 "\r\nMaruchan Ramen Beef Flavor 18 oz. Noodle Soup\r\n",
 "\r\nMaruchan Ramen Chili Flavor 3 oz. Noodle Soup\r\n",
 .
 .
 .
 "\r\nRamen Less Sodium Beef - 3 oz.\r\n"]
```

Since the order of product names and prices will be in the same order, we can check for the name of the noodles, then use that index to find the price.


```
for index, name in enumerate(cleaned_titles):
    if "Maruchan Ramen Beef Flavor 3 oz." in name:
        price_index = index
```

Now we create a list of tag objects representing the prices and pull out the index we found above. We could use tag.text to directly access the price, but for the sake of the example let's use a regular expression.

```
>>> prices = soup.find_all(class_ = "product_productprice")
>>> our_price = str(prices[price_index])
>>> our_price
'<div class="product_productprice"><b><font class="text
colors_text">Price</font> $6.99 / Case of 24</b></div?>'
>>> final_beef_price = re.search(r"\$\d*\.\d\d", our_price).group()
>>> final_beef_price
'$6.99'
```

We are done! We can compile this process into one script, and be able to run it directly from our command prompt to instantly give us the up-to-date ramen price.

Selenium

To scrape data from websites using dynamic rendered content, you can use the [Selenium module](#). This is our [Selenium handout](#).

References

Requests documentation: <https://requests.readthedocs.io/en/latest/user/quickstart/>

Requests helpful guide: <https://realpython.com/python-requests/>

RegEx documentation: <https://docs.python.org/3/library/re.html>

Helpful RegEx guide: <https://developers.google.com/edu/python/regular-expressions>

BeautifulSoup documentation: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Other web scraping modules that we don't discuss, but could be useful/interesting:

- <https://mechanize.readthedocs.io/en/latest/>
- <https://docs.scrapy.org/en/latest/>

Remember that Google is your friend!! Errors with these modules can usually be fixed using StackOverflow answers or the documentation.